1. What is OS? What are its important functions?
2. What is system call? How it is executed?
    * Functions exposed by the kernel.
    * Program --> SysCall wrapper --> Software Interrupt --> SWI Handler --> System call table --> SysCall Impl
3. Explain terms: Multi-programming, Multi-tasking, Multi-threading, Multi-processing and Mutli-user?
    * Multi-programming: Loading multiple programs in main memory.
        * Better CPU utilization.
    * Multi-tasking: Sharing CPU time among multiple tasks present in main memory and "ready" for execution.
        * Process based ========== (RESEARCH AGAIN)==========
        * Thread based -- Multi-threading:========== (RESEARCH AGAIN)==========
    * Multi-processing:
        * Parallel processing -- Multiple CPUs are utilized for execution program.
        * Higher throughput
    * Multi-user:
        * Multiple users executing multiple processes concurrently.
4. Explain Linux kernel design (monolithic or modular).
    * /boot/vmlinuz --> Linux kernel -- Monolithic -- Static components
        * System calls, Scheduler, Process/Thread managment, Memory managment, HAL, ...
        * make bzImage --> vmlinux --> vmlinuz
    * /lib/modules/kversion --> Kernel modules --> Modular --> Dynamic components
        * Device drivers, File System managers, ...
        * make image --> *.ko
5. Explain process life cycle.
    * OS process life cycle: New, Ready, Running, Waiting, Terminated.

    * Linux process life cycle: Running/Runnable, Sleep (Interruptible or Non-interruptible), Zombie, Dead.

6. What are Linux IPC mechanisms? Explain any three with diagram.
    * Signals -- kill() & signal() -- Signal delivery.
    * Shared memory -- syscalls, internal diagram.
    * Message queue -- syscalls (blocking -- msgget()) -- packet-based / bi-directional
    * Pipe -- blocking (full, empty) -- stream-based / uni-directional -- when writer/reader is stopped?
    * Socket

7. What is difference between process and thread? How can you create them in Linux program?
    * fork() -- syscall
    * pthread_create() -- POSIX lib function

8. What is difference between semaphore, mutex and spinlocks? How can you create them in Linux program?
    * Semaphore -- Counter -- P() op & V op()
        * Sync mechanism -- Counting (resource counter) and Binary (mutual excl and event flag)
        * Locked process/thread --> Blocked/Watinging state
    * Mutex -- Mutual exclusion
        * Locked --> Owner --> Unlock
        * Locked process/thread --> Blocked/Watinging state
    * Spinlock -- Hw sync mechanism -- bus holding instructions -- availble only in kernel space
        * Locked process/thread --> Running state
        * Semaphore/Mutex in internally use Spinlock

9. What is file? Which system calls are used to manipulate files in Linux? Write their prototypes and explain.
    * open(), close(), read(), write() and lseek()

10. What is paging? What is page fault and how it is handled?
    * Process memory -- Pages (Logical pages) --> RAM -- Frames (Physical pages).
    * Tracking is done in Page table --> Per process --> 2/3-level paging.

    * Page fault -- Access page that is not in main memory (RAM).
    * OS -- Page fault exception handler
11. What is difference between starvation and deadlock?
    * starvation -- process is ready to execute but not getting CPU time.
        * Due to less priority.
    * deadlock -- processes involved in deadlock are waiting/blocked state.
        * no preemption, mutual exclusion, hold & wait, circular wait.
12. How vfork() system call works? What is copy-on-write?
    * vfork() -- virtual fork
        * child memory is overlays with parent
        * parent is suspended
        * parent thread execute child until exec() or exit()
    * COW -- logical copy of addr space in fork()
        * Page table is copied, but point to same pages in RAM
        * When parent/child try to modify, actual copy of page is created and updated in respective page table.
13. Explain OS booting. Explain Linux booting.
    * Linux Booting: Grub stages, Linux runlevels, init vs systemd.
14. How many Linux run-levels are there? Which features are enabled in each runlevel?
    * 1 -- single user mode
    * 2 -- multi-user mode
    * 3 -- networking mode
    * 5 -- graphical mode
    * 0 -- shutdown, 6 -- reboot
15. Explain regex commands and wildcard characters.
    * grep, egrep
    * *, ., ?, +, [..], ...
16. Which Linux command is used to kill all running instances of the same program.
    * pkill -9 java

17. How security is implemented in Linux file systems? Tell commands related to it.
    * ugo -- rwx rwx rwx
    * chmod and chown
18. How redirection and pipe in Linux commands work?
    * output redirection > vs error redirection 2>
19. Symbolic Link vs Hard Link
20. Linux file system hierarchy.

==========

1. What is difference between microcontroller and microprocessor?
2. What is difference between I/O mapped I/O and Memory mapped I/O?
    * IO mapped IO -- separate bus/address space for IO devices.
        * Special IO instructions e.g. x86 IN/OUT
        * Special IO signal/pin e.g. IO/M
    * Memory mapped IO -- bus/address space is shared between IO & memory
        * Same instructions as of memory.
        * e.g. ARM
3. What is pipeline? What are limitations? Explain pipeline in ARM architecture.
    * Parallel processing of instructions.
    * Multiple blocks executing independently/parallely but cascaded.
    * Fetch --> Decode --> Execute.
    * Instructions are NOT stored, they are processed.
4. What is ARM 7 pipeline? What are limitations of pipeline?
    * Fetch --> Decode --> Execute.
    * Pipeline hazards --> Control, Data, Structural.
5. What is Difference between Fast GPIO and Legacy GPIO?
    * ARM7 --> Fast/Enhanced GPIO vs Legacy GPIO
        * Fast GPIO -- ARM core bus -- high speed
        * Legacy GPIO -- Peripheral bus -- low speed

        * Fast GPIO -- Word, Half-word or Byte accessible
        * Legacy GPIO -- Word accessible
        * Fast GPIO -- Also have Mask register
        * Legacy GPIO -- No mask register
    * ARM CM3 --> Fast/Enhanced GPIO
6. What is Difference between Von-Neumann and Harvard?
    * Buses
    * Instructions
7. Explain ARM Cortex-M modes? Explain register banking.
    * Modes: Thread (OS/Program execution) vs Handler (Exception handling)
    * Register banking: MSP vs PSP
7. Why FIQ is faster than IRQ -- ARM7 / ARM CA?
    * FIQ priority is higher than IRQ.
    * FIQ has additional GPR. So no need to save context.
    * Last entry in EVT. So jump instruction can be skipped.
8. Explain protocols in detail: RS-232, I2C, SPI?
    * Protocol explanation
        * Physical charcteristics
            * Wires (Simplex, Half-duplex and Full-duplex)
            * Voltage/current levels
            * Frequency/Speed
            * Diagram -- Wiring / Block
            * Peer to peer or Bus protocol.
        * Logical charcteristics
            * Frame: START .... STOP
        * Error conditions
    * SPI
        * Bus protocol
        * Slave selection -- GPIO -- Number of GPIO pins (if many slaves).
        * Daisy chaining -- Slave selection with less number of pins.
    * I2C
        * Bus protocol
        * Slave selection -- I2C address (7-bit).

9. What is wired AND in I2C? What is clock stretching? What is bus arbitration?
    * If any device pull line low, effective line level will be low -- Wired AND
    * clock stretching -- receiver pull clock line low to pause further data transmission from transmitter.
    * arbitration -- multiple masters start at same moment, one with first 0 wins arbitration.
10. Which programmer you have used for ARM and AVR?
    * Programmer hardware: * STlink/V2, JTAG, ...
    * Programmer software: * STM32CubeIDE (STM32 Programmer), Flash Magic, AvrDude, ...
11. What are ARM Processor states?
    * CM --> Thumb and Debug
    * CA --> ARM, Thumb and Jazelle
12. Compare ARM, AVR, PIC and 8051 controllers?
    * Mazidi -- Chapter 1.
13. In 8086 architecture what is difference in SP and BP and SS registers?
    * SP -- Stack Pointer -- Top of stack (Full descending).
    * SS -- Stack Segment -- Bottom of stack (Starting/Base address).
    * BP -- Base Pointer -- Base address of current stack frame (function activation record). Local vars and Arguments are accessed w.r.t. BP.
14. Explain watchdog timer and brown out detection?
    * WDT -- To check if task is completed in expected time duration. If not, reset the processor.
    * BOD -- To check if power/voltage is dropped. If dropped, reset the processor.
15. Explain the Instruction Set for ARM and AVR in detail?
16. Give overview of CAN protocol? Where CAN find its main usage.
    * Physical characteristics
    * Logical characteristics
17. What is the use of the AMBA interface and explain it in detail.

    * Advanced Microcontroller Bus Interface.
    * Timer based -- Bus multiplexing

18. What are the types of addressing modes in ARM?
    * Immediate
    * Register
    * Direct addressing
    * Indirect addressing
    * ...

19. What is Translation Lookaside Buffer (TLB)?
    * Paging MMU -- OS

20. What is single issue multiple data (SIMD) processing?
    * QADD8 --- four 8-bit additions in one instructions.

21. How will you allow Thumb C code to call the ARM assembly Code?
    * __asm("BLX label")
        * X -- exchange state
        * last bit of address -- state -- ARM state (0) or Thumb state (1)

22. What is the use of 'SWI' in ARM assembly?
    * SWI -- Software Interrupt -- System call.

23. Tell about the Exception Handling in ARM processor. What does the ARM Core do automatically for every exception?
    * Cortex A
    * Cortex M

24. Explain memory map in ARM architecture.
    * 32-bit arch -- 4 GB address space
        * Flash, SRAM, Bit Band, IO Peripherals, Private Peripherals

25. Explain difference between ARM or Thumb state?
    * ARM state -- 32 bit instructions -- PC (2 bits X)
    * Thumb state -- 16 bit instructions -- PC (1 bit X)

26. Explain about the bits in CPSR register.
    * Cortex A -- CPSR -- ALU, Modes (5-bit)
    * Cortex M -- xPSR -- ALU, Base Intr

27. What is an interrupt? Write a C program to handle external interrupt for an ARM chip.
    * Cortex-M3 exception handling -- code

    * IVT -- array of function pointers
    * All functions are defined in assembly (startup.S) -- weak functions -- #pragma / .weak
    * Programmer will define the intended ISR function.

28. Explain ARM Cortex-M3 architecture.
    * States, Modes, Priv Levels, Registers
    * Exceptions (if required)